

AMAR TIWARI · UPDATED 4 DAYS AGO

25

Code

Download

Student Performance Dataset

5000 student records on productivity, burnout, focus, and exam performance.

Data Card

Code (8)

Discussion (0)

Suggestions (0)

Dataset Overview

About Dataset

Student Performance Dataset (5000 Records)

This dataset contains 5000 synthetic student records designed to analyze the relationship between study habits, digital behavior, mental health, and academic performance.

It is ideal for machine learning, data analysis, regression modeling, classification tasks, and multi-target prediction projects.

Usability

10.00

License

CC0: Public Domain

Expected update frequency

Never

Tags

We will create a model to predict a student’s exam score.

First we need a Compute instance.

Automated ML

Designer

Prompt flow

Assets

Data

Jobs

Components

Pipelines

Environments

Models

Endpoints

Manage

Compute

Monitoring

Compute

Compute instances

Compute clusters

Kubernetes clusters

Attached computes

Serverless

Choose from a selection of CPU or GPU instances preconfigured with popular tools such as VS Code, Jupyter, and more. [Learn more about compute instances](#)

+ New

Refresh

Start

Stop

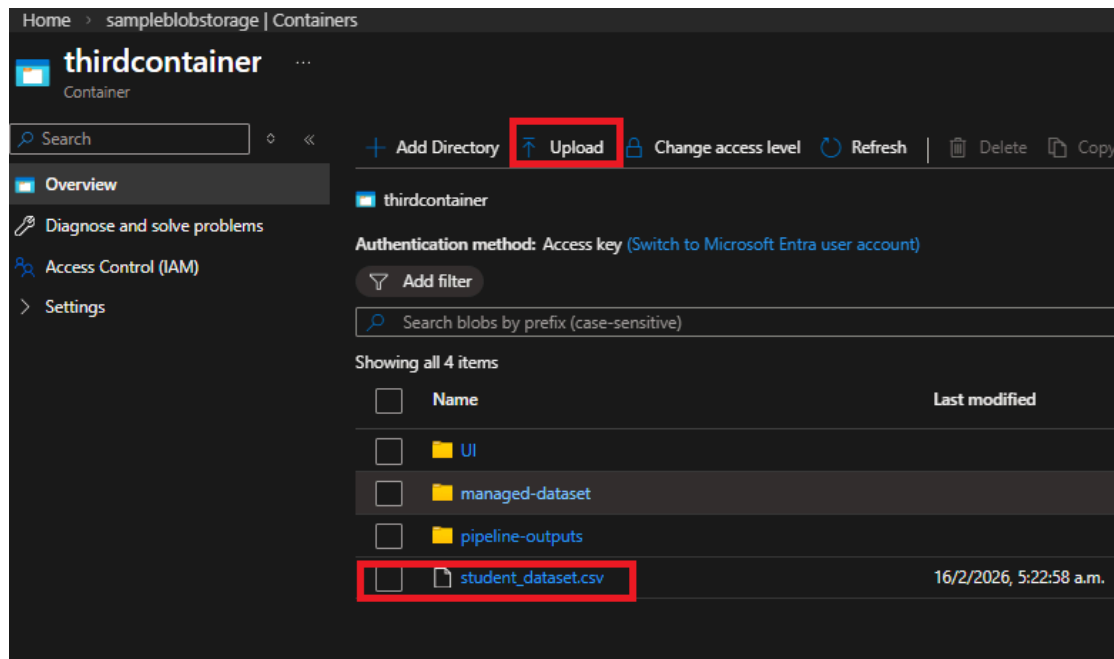
Restart

Schedule and idle settings

Search

Name ↑	State	Idle shutdown
samplecomputeinstance	Running	1 hour
simplecomputeinstance2	Stopped	1 hour

Upload the dataset in a Container instance



Go to Notebooks and create one, put the next imports

```
from azureml.core import Workspace, Dataset, Datastore, Experiment
from azure.ai.ml import MLClient
from azure.identity import DefaultAzureCredential

import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.compose import ColumnTransformer,
TransformedTargetRegressor
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
```

set up your workspace.

```
ws = Workspace.from_config()
```

list datastores and check if you can see your datastore.

```
for ds in ws.datastores:
    print(ds)
```

```
thirddatastore
```

Good, now we need to create our client.

```
ml_client = MLClient(  
    DefaultAzureCredential(),  
    subscription_id="...",  
    resource_group_name="...",  
    workspace_name="..."  
)
```

Retrieve your data asset.

```
data_asset = ml_client.data.get('student_data', version='1')
```

Read your csv.

```
df = pd.read_csv(data_asset.path)  
print(df.head())
```

Verify null values.

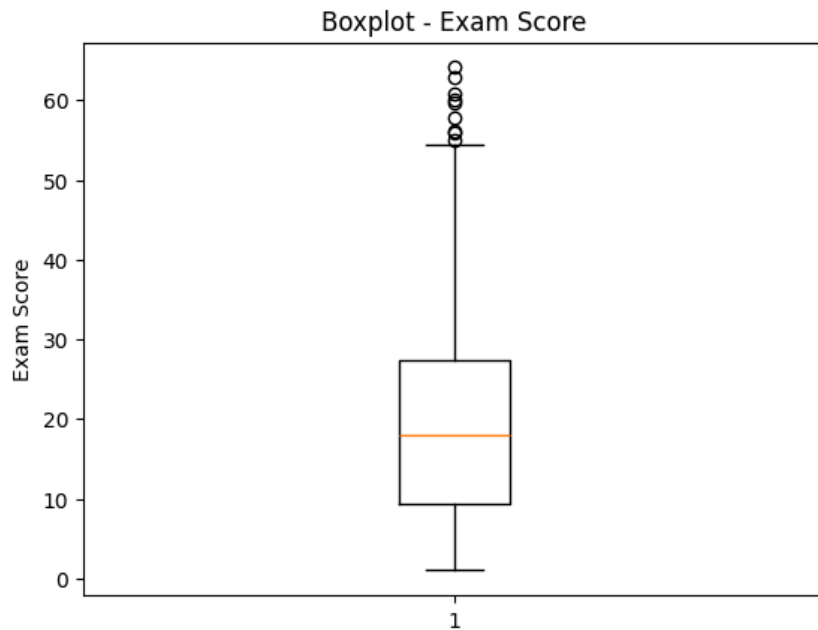
```
df.isnull().sum()
```

Verify data types.

```
df.dtypes
```

Plot a boxplot to see data distribution.

```
import matplotlib.pyplot as plt  
  
plt.figure()  
plt.boxplot(df["exam_score"])  
plt.title("Boxplot - Exam Score")  
plt.ylabel("Exam Score")  
plt.show()
```



Begin an experiment to log all outputs in a job.

```
experiment = Experiment(ws, 'StudentPerfomanceModel')
```

Initialize it.

```
run = experiment.start_logging()
```

Now we can remove outlier values and get 50% central data.

```
Q1 = df["exam_score"].quantile(0.25)
Q3 = df["exam_score"].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

df_clean = df[(df["exam_score"] >= lower) & (df["exam_score"] <=
upper)]

print("Original:", len(df))
print("Without outliers:", len(df_clean))
```

You will get:

Original: 5000

Without outliers: 4991

Ok, now it's time to define our pipeline.

```
categorical_cols = ["gender", "academic_level", "internet_quality"]
numerical_cols = df_clean.drop(columns=["exam_score"]).select_dtypes(
```

```

        include=["int64","float64"]
    ).columns

preprocessor = ColumnTransformer(
    transformers=[
        ("cat", OneHotEncoder(drop="first"), categorical_cols),
        ("num", StandardScaler(), numerical_cols)
    ]
)

regressor = TransformedTargetRegressor(
    regressor=LinearRegression(),
    transformer=StandardScaler()
)

pipeline = Pipeline([
    ("preprocessor", preprocessor),
    ("model", regressor)
])

X = df_clean.drop(columns=["exam_score"])
y = df_clean["exam_score"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

pipeline.fit(X_train, y_train)

```

After that we can evaluate and get our results.

```

from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score

y_pred = pipeline.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("MAE:", mae)
print("MSE:", mse)
print("RMSE:", rmse)
print("R2:", r2)

```

Now log them.

```
run.log('mean absolute error', mae)
run.log('mean squared error', mse)
run.log('root mean squared error', rmse)
run.log('r2 score', r2)
```

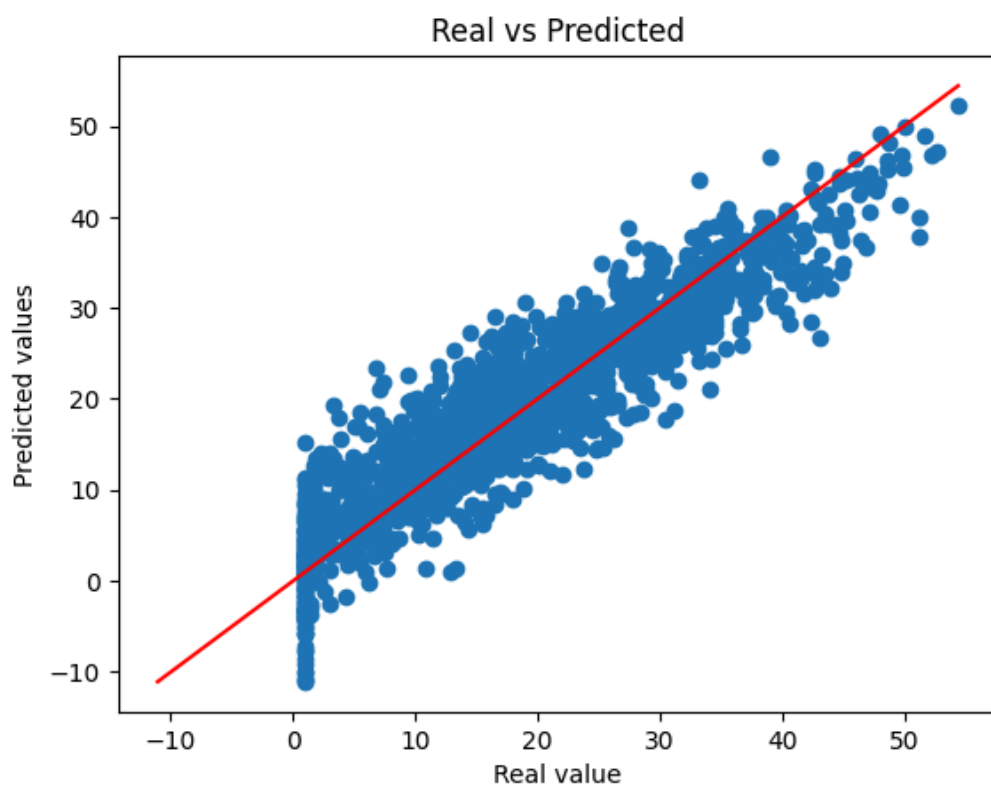
We will graph it to see how it looks.

```
plt.figure()
plt.scatter(y_test, y_pred)

min_val = min(y_test.min(), y_pred.min())
max_val = max(y_test.max(), y_pred.max())

plt.plot([min_val, max_val], [min_val, max_val], color='red')

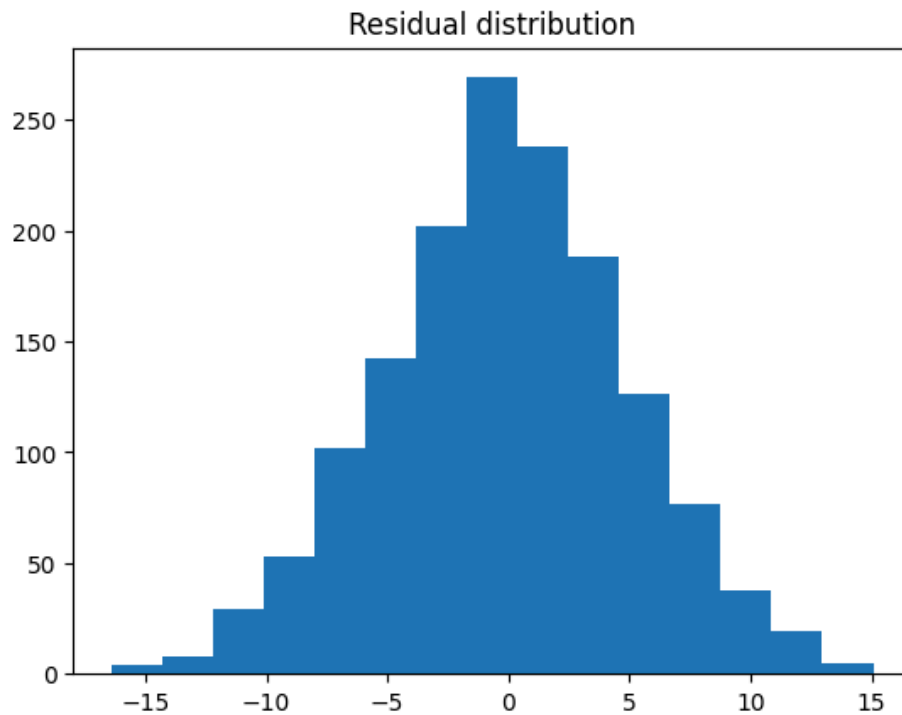
plt.xlabel("Real value")
plt.ylabel("Predicted values")
plt.title("Real vs Predicted")
plt.show()
```



Now graph the residual distribution

```
plt.figure()
plt.hist(residuals, bins=15)
```

```
plt.title("Residual distribution")
plt.show()
```



Finally complete the run.

```
run.complete()
```

Our results are:

orange_duck_wcs7mcf Completed

Overview Metrics Images Child jobs Outputs + logs Code Monitoring

Refresh Resubmit Debug and monitor Register model Cancel Delete Compare

Properties

Status Completed	Script name --
Created on Feb 16, 2026 6:23 AM	Created by Ricardo Robledo
Start time Feb 16, 2026 6:23 AM	Experiment StudentPerformanceModel
Duration 25.17s	Arguments None
Compute duration 25.17s	Registered models None
Compute target local	See all properties Raw JSON
Name	

Tags

No tags

Metrics

mean absolute error 3.852718	r2 score 0.8382284
mean squared error 23.70089	root mean squared error 4.868356

Description

Click edit icon to add a description